M12 Learner Book

Distributed Processing with Spark / PySpark | Release Candidate 1 | 18 controlled lessons

M12-L01 - Why Spark exists: driver, executors, partitions and tasks

Learning objective

Explain the distributed execution model at a beginner-usable level.

Why this matters in Data Engineering

Scale changes how work is split, scheduled and recovered.

Understand

• Driver coordinates application logic.

• Executors perform work in distributed deployments.

• Partitions are units of data parallelism; tasks operate on partitions.

Try / inspect

```
local[2] -> SparkSession -> defaultParallelism
```

Common failure

Treating tasks as rows or thinking local mode teaches cluster sizing.

Your turn - independent proof

Explain driver/executor/partition/task using the packaged lab.

Move on when

You can trace one simple action from driver plan to partition tasks.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L02 - PySpark setup, SparkSession and laptop-safe local mode

Learning objective

Start/stop a resource-conscious Spark session and verify versions.

Why this matters in Data Engineering

A learner needs reproducible setup without turning installation into the lesson.

Understand

• Use isolated environment.

• Current stable Spark docs are 4.2.0 as verified 2026-08-27.

• Course local profile is teaching-safe, not production tuning.

Try / inspect

```
SparkSession.builder.master("local[2]").getOrCreate()
```

Common failure

Running every heavy stack simultaneously or treating 1g/4 partitions as production defaults.

Your turn - independent proof

Verify Java/Python/PySpark and create/stop a session when PySpark is installed.

Move on when

You can verify the environment and explain the local profile limits.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L03 - DataFrames and explicit schemas

Learning objective

Create typed DataFrames and reason about rows/columns/grain.

Why this matters in Data Engineering

Schema mistakes propagate into every transformation.

Understand

• Prefer explicit schema for controlled pipelines.

• DataFrame operations are declarative.

• State grain before joins/aggregations.

Try / inspect

```
df.printSchema(); df.select("order_id","qty")
```

Common failure

Relying on inference for ambiguous production inputs or losing grain.

Your turn - independent proof

Load orders with an explicit schema and state one-row meaning.

Move on when

Schema and grain are both explicit and validated.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L04 - Spark SQL and expression equivalence

Learning objective

Use DataFrame and SQL interfaces without changing business meaning.

Why this matters in Data Engineering

Teams often mix APIs; correctness should survive interface choice.

Understand

• Register temp views deliberately.

• Spark SQL and DataFrame APIs share the optimizer for many operations.

• Validate equivalent outputs rather than assuming equivalence.

Try / inspect

```
df.createOrReplaceTempView("orders")
spark.sql("select ...")
```

Common failure

Writing two versions that differ in filter/grain and calling them equivalent.

Your turn - independent proof

Implement one filter/select both ways and compare results.

Move on when

You can explain the same transformation in SQL and DataFrame form.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L05 - Lazy transformations and actions

Learning objective

Explain when Spark actually executes work.

Why this matters in Data Engineering

Lazy evaluation enables optimization but confuses beginners debugging pipelines.

Understand

• Transformations build a logical plan.

• Actions trigger jobs.

• Repeated actions may recompute unless data is persisted/reused.

Try / inspect

```
filtered=df.filter(...); filtered.count()
```

Common failure

Assuming a transformation has already read/written all data.

Your turn - independent proof

Identify transformations vs actions in lab output.

Move on when

You can predict which line triggers execution.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L06 - Jobs, stages, tasks and physical plans

Learning objective

Read plan boundaries rather than treating Spark as a black box.

Why this matters in Data Engineering

Operational debugging needs evidence of what the engine plans and runs.

Understand

• An action can create one or more jobs.

• Shuffle boundaries commonly separate stages.

• Tasks run per partition within stages.

• explain() exposes physical-plan evidence.

Try / inspect

```
df.groupBy("region").sum("amount").explain()
```

Common failure

Inventing performance claims without plan/UI evidence.

Your turn - independent proof

Use explain and identify at least one exchange/stage-driving operation.

Move on when

You can connect a plan operator to execution behavior.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L07 - repartition vs coalesce

Learning objective

Control partition count deliberately.

Why this matters in Data Engineering

Partition changes affect parallelism, shuffle cost and output file count.

Understand

• repartition can increase/decrease and normally shuffles.

• coalesce is mainly a narrow reduction.

• Measure rather than tune by superstition.

Try / inspect

```
df.repartition(4); df.coalesce(1)
```

Common failure

Using coalesce(1) on large production output because one file looks tidy.

Your turn - independent proof

Compare partition counts and plan changes.

Move on when

You can choose a method based on purpose and scale.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L08 - Aggregations and shuffle cost

Learning objective

Recognize wide transformations and shuffle evidence.

Why this matters in Data Engineering

Group/aggregate at scale can dominate runtime/network cost.

Understand

• Grouping redistributes keys when needed.

• Exchange/shuffle is expensive relative to local map-like work.

• Correctness and grain come before tuning.

Try / inspect

```
df.groupBy("region").agg(sum("gross"))
```

Common failure

Optimizing away a shuffle by changing the business result.

Your turn - independent proof

Locate Exchange/shuffle evidence and reconcile aggregation total.

Move on when

You can explain why the shuffle occurs and prove output correctness.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L09 - Joins and broadcast strategy

Learning objective

Choose join strategy from relation size and evidence.

Why this matters in Data Engineering

Bad joins can be wrong or expensive; broadcast is not a magic switch.

Understand

• Join keys must preserve intended grain.

• Broadcasting a genuinely small relation can avoid a large shuffle.

• Large/uncertain relations should not be blindly broadcast.

Try / inspect

```
fact.join(broadcast(dim),"product_id")
```

Common failure

Broadcasting a huge fact table or ignoring duplicate dimension keys.

Your turn - independent proof

Compare a normal join and a safe small-dimension broadcast plan.

Move on when

You can defend key uniqueness and broadcast choice.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L10 - Join correctness, fan-out and reconciliation

Learning objective

Detect row multiplication independently of performance.

Why this matters in Data Engineering

A fast wrong join is still wrong.

Understand

• Declare expected input/output grain.

• Check dimension-key uniqueness before join.

• Reconcile row counts/totals after join.

Try / inspect

```
before=count; after=count; totals_before==totals_after
```

Common failure

Treating a row-count increase as harmless without explaining grain.

Your turn - independent proof

Inject/inspect a duplicate dimension key and show the fan-out check.

Move on when

You can prove the join preserves intended business identity.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L11 - Data skew

Learning objective

Recognize hot keys and mitigation options.

Why this matters in Data Engineering

A few heavy partitions can dominate stage completion.

Understand

• Skew means uneven work/data distribution.

• Measure key frequencies and task durations.

• Mitigations include salting, preaggregation, repartitioning or business redesign depending on case.

Try / inspect

```
groupBy("key").count().orderBy(desc("count"))
```

Common failure

Assuming more partitions automatically fixes a hot key.

Your turn - independent proof

Show the HOT key dominates and propose two evidence-based mitigations.

Move on when

You can distinguish skew from general slowness.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L12 - Adaptive Query Execution (AQE)

Learning objective

Explain what AQE can and cannot adapt at runtime.

Why this matters in Data Engineering

Modern Spark can revise parts of physical execution, but data design still matters.

Understand

• AQE is enabled by default in modern Spark SQL.

• It can coalesce shuffle partitions and adapt supported join/skew decisions.

• It does not repair wrong grain or bad business logic.

Try / inspect

```
spark.conf.get("spark.sql.adaptive.enabled")
```

Common failure

Claiming AQE removes all need for partition/join reasoning.

Your turn - independent proof

Inspect AQE setting and compare initial/final plan where exposed.

Move on when

You can describe AQE as adaptive execution, not correctness insurance.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L13 - Caching and persistence

Learning objective

Cache reused data only when reuse justifies memory/storage cost.

Why this matters in Data Engineering

Unnecessary caching wastes executor memory and can slow jobs.

Understand

• cache/persist is lazy until action.

• Reuse count and recomputation cost matter.

• Unpersist when no longer needed.

Try / inspect

```
df.cache(); df.count(); ...; df.unpersist()
```

Common failure

Caching every DataFrame by default.

Your turn - independent proof

Materialize a cache and explain when it helps vs hurts.

Move on when

You can justify cache from reuse and resource evidence.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L14 - Parquet I/O and partition pruning

Learning objective

Use columnar storage efficiently from Spark.

Why this matters in Data Engineering

Distributed processing and storage layout interact.

Understand

• Parquet supports column pruning and predicate-related skipping.

• Partitioned directories can prune whole partitions.

• Too many partitions/files can create metadata overhead.

Try / inspect

```
spark.read.parquet(path).filter("order_date=...")
```

Common failure

Equating partition pruning with row-level filtering or overpartitioning output.

Your turn - independent proof

Write/read partitioned Parquet and inspect output tree plus plan.

Move on when

You can explain file layout and read plan together.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L15 - Built-in functions vs Python UDFs

Learning objective

Prefer optimized built-ins when they express the logic.

Why this matters in Data Engineering

Cross-language serialization can make Python UDFs slower and less optimizable.

Understand

• Use Spark SQL functions first.

• UDFs are appropriate only when needed.

• Correctness tests are required either way.

Try / inspect

```
withColumn("gross", col("qty")*col("unit_price"))
```

Common failure

Using a Python UDF for simple arithmetic/string/date operations.

Your turn - independent proof

Rewrite one UDF-like task with built-ins and compare the plan.

Move on when

You can state when a UDF is justified.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L16 - Driver safety: collect, toPandas and bounded inspection

Learning objective

Keep large data distributed and bound what reaches the driver.

Why this matters in Data Engineering

Driver OOM is a common avoidable failure.

Understand

• collect() brings all rows to the driver.

• limit/show/take are safer for inspection.

• toPandas also centralizes data and needs a size boundary.

Try / inspect

```
df.limit(20).show(truncate=False)
```

Common failure

Unbounded collect()/toPandas on unknown data size.

Your turn - independent proof

Repair the packaged bad collect TODO with a bounded alternative.

Move on when

You can explain where data moves and why the driver remains safe.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L17 - Testing and debugging Spark transformations

Learning objective

Make transformations testable on tiny deterministic data before scaling.

Why this matters in Data Engineering

Fast local tests catch business logic defects earlier than cluster debugging.

Understand

• Separate pure transformation logic from I/O.

• Use tiny fixtures with known outputs.

• Use explain, logs, UI and counts to diagnose execution.

Try / inspect

```
transform_orders(spark.createDataFrame(tiny_rows))
```

Common failure

Only testing by eyeballing a large output file.

Your turn - independent proof

Complete the transform TODO/test and record one debugging path.

Move on when

A clean test proves transformation behavior independently of file I/O.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M12-L18 - Integrated PySpark pipeline and distributed-processing checkpoint

Learning objective

Build and defend a complete Spark pipeline with correctness and plan evidence.

Why this matters in Data Engineering

Job readiness requires integration: schema, transforms, joins, aggregation, storage and safe operation.

Understand

• Reconcile 18 source rows -> 16 completed rows.

• Expected gross sales total is 25230.00 in the packaged dataset.

• Explain one job/stage/shuffle/join decision with evidence.

• Keep the laptop-safe profile separate from production sizing claims.

Try / inspect

```
python -m src.pipeline_TODO
validate outputs + explain plan
```

Common failure

Calling the pipeline complete because it runs once without reconciliation.

Your turn - independent proof

Build pipeline_TODO independently and defend correctness plus one execution-plan decision.

Move on when

Outputs reconcile and you can explain how Spark executed the key transformations.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.